BACKEND DE APLICACIONES · UTN FRC · WINDOWS

Guía rápida para el parcial

Lo que hay que tener listo y saber antes del parcial: JDK, Maven, la estructura del proyecto, leer cualquier CSV y hacer búsquedas con Streams. Con VS Code ya instalado.

01 · Instalación

Instalar el JDK

Compila y ejecuta el código Java. Versión 21.

1. Descargar: **oracle.com/java/technologies/downloads/#java21** → instalador `.exe` Windows.
2. Instalar (Next → Next → Finish). Queda en `C:\Program Files\Java\jdk-21`.
3. Variables de entorno del sistema → Nueva: `JAVA_HOME` = `C:\Program Files\Java\jdk-21` → editar `Path` → agregar `%JAVA_HOME%\bin`.
4. Cerrar y volver a abrir la terminal.

Verificar:

```terminal
java -version
javac -version
```

---

02 · Instalación

Instalar Maven

Gestiona el proyecto: crea la estructura, compila y ejecuta. Necesita el JDK ya instalado.

1. Descargar: **maven.apache.org/download.cgi** → Binary zip archive.
2. Extraer en una carpeta fija que **no vayas a borrar nunca** (ej. `C:\Herramientas\maven`) — no dentro de la carpeta de un proyecto puntual: si esa carpeta se borra algún día, se lleva Maven puesto y se rompe para todos tus proyectos.
3. Configurar variables de entorno del sistema (tecla Windows → escribir "variables de entorno" → "Editar las variables de entorno del sistema" → botón "Variables de entorno..."):
    a. En "Variables del sistema" → "Nueva..." → Nombre: `MAVEN_HOME`, Valor: la carpeta donde extrajiste Maven.
    b. Buscar la variable `Path` en la misma lista → "Editar..." → "Nuevo" → escribir `%MAVEN_HOME%\bin` (o "Examinar..." y elegir la carpeta `bin` a mano — da igual, funciona lo mismo).
    c. Aceptar en todas las ventanas.
4. Cerrar **todas** las terminales y VS Code, y volver a abrir.

Verificar:

```
terminal
mvn -v
```

---

03 · Configuración

Extensiones Java en VS Code

Sin esto, VS Code abre los `.java` como texto plano: no autocompleta ni te deja correr el `main` con un botón.

1. `Ctrl+Shift+X` → instalar **"Extension Pack for Java"** (Microsoft).
2. `File → Open Folder` → la carpeta que tiene el `pom.xml`.

Terminal integrada: `Ctrl + ñ` — ahí van todos los comandos `mvn`.

---

04 · Proyecto

Generar el proyecto

Un proyecto Maven no se arma a mano: se genera a partir de un **arquetipo** (una plantilla) que crea el `pom.xml` y toda la estructura de carpetas `src/` lista para usar. Corré esto y respondé lo que te va preguntando:

```
terminal
mvn archetype:generate
```

TE PREGUNTA	RESPONDÉS
Filtro / número de arquetipo	Escribí `quickstart` → Enter → elegí `maven-archetype-quickstart`
Número de versión	Enter (default)
`groupId`	`ar.edu.utnfc.backend`
`artifactId`	nombre del proyecto, ej. `parcial-backend`
`version`, `package`, confirmación	Enter, Enter, Enter

```
estructura generada
parcial-backend/
├─ pom.xml
└─ src/main/java/ar/edu/utnfc/backend/App.java
```

Fijate la versión de Java en el `pom.xml`: tiene que decir 17 o 21 (`<maven.compiler.release>`, o `source` y `target` como en el apunte 03). Si dice 1.7 u 1.8 cambialo a 21; si no, no compilan `var`, el `switch` con `->`, los `record` ni `.toList()`.

Para correrlo con `mvn exec:java`, agregá dentro de `<build><plugins>` en el `pom.xml`:

pom.xml

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.1.0</version>
  <configuration>
    <mainClass>ar.edu.utnfc.backend.app.Main</mainClass>
  </configuration>
</plugin>
```

---

05 · Proyecto

Tres paquetes

Dentro de `src/main/java/ar/edu/utnfc/backend/`, tres carpetas:

PAQUETE	QUÉ VA ACÁ
`app`	El `main`: orquesta todo.
`entidades`	POJOs — una fila del CSV convertida a objeto.
`servicios`	Lectura del CSV + búsquedas/filtros.

---

06 · Proyecto

Ejemplo funcionando

Orden real de armado: **1)** las entidades — en el parcial normalmente ya tenés el DER (o incluso las clases JPA armadas desde el pre-enunciado); si hay que crearlas, van en `entidades/`. **2)** los servicios con la lógica. **3)** el `main` en `app/` que llama a todo. Toda entidad tiene las mismas partes — quedan marcadas como comentario para que sirva de plantilla con cualquier clase que te den:

1 — entidades / Venta.java

entidades/Venta.java

```
// 1. paquete: siempre "entidades"
package ar.edu.utnfc.backend.entidades;

// 2. imports que necesite la clase
import java.time.LocalDate;

public class Venta {

    // 3. atributos: uno por cada columna del CSV / campo del DER
    private String cliente;
    private String articulo;
    private String categoria;
    private double precioUnitario;
    private int cantidad;
    private LocalDate fecha;

    // 4. constructor: recibe todos los atributos y los asigna con "this."
    public Venta(String cliente, String articulo, String categoria,
                 double precioUnitario, int cantidad, LocalDate fecha) {
        this.cliente = cliente;
        this.articulo = articulo;
        this.categoria = categoria;
        this.precioUnitario = precioUnitario;
        this.cantidad = cantidad;
        this.fecha = fecha;
    }

    // 5. getters: uno por atributo (sin setters si no se van a modificar)
    public String getCliente() { return cliente; }
    public String getArticulo() { return articulo; }
    public String getCategoria() { return categoria; }
    public double getPrecioUnitario() { return precioUnitario; }
    public int getCantidad() { return cantidad; }
    public LocalDate getFecha() { return fecha; }

    // 6. métodos propios: cálculos que no vienen directo del CSV
    public double getTotal() { return precioUnitario * cantidad; }

    // 7. toString: para poder imprimir el objeto entero con
System.out.println(venta)
    @Override
    public String toString() {
```

```
        return cliente + " - " + articulo + " x" + cantidad + " ($" + getTotal() +
")";
    }
}
```

`@Override` no es "`@toString`": es una anotación de Java que avisa "este método reemplaza uno que ya existe en la clase padre" (acá, en `Object`). Por eso aparece sobre `toString()`, `equals()` y `hashCode()` — todo objeto los hereda de `Object`, y esto es solo redefinirlos. No es obligatoria, pero si hay error de tipeo en la firma del método, el compilador avisa.

Opción A vs. Opción B: a mano, o con Lombok

Todo lo de arriba (constructor, getters, `toString`) se puede escribir a mano — como venís haciendo — o generarlo solo con **Lombok**, una librería que agrega esos métodos en el momento de compilar, a partir de anotaciones puestas arriba de la clase. Las dos opciones compilan y funcionan exactamente igual; Lombok solo evita tipear lo repetitivo.

OPCIÓN A · A MANO

entidades/Venta.java

```
package ar.edu.utnfc.backend.entidades;

import java.time.LocalDate;

public class Venta {
    private String cliente;
    private String articulo;
    private double precioUnitario;
    private int cantidad;
    private LocalDate fecha;

    public Venta(String cliente, String
articulo,
                 double precioUnitario,
int cantidad,
                 LocalDate fecha) {
        this.cliente = cliente;
        this.articulo = articulo;
        this.precioUnitario =
precioUnitario;
        this.cantidad = cantidad;
        this.fecha = fecha;
    }

    public String getCliente() { return
cliente; }
    public String getArticulo() {
return articulo; }
    public double getPrecioUnitario() {
return precioUnitario; }
    public int getCantidad() { return
cantidad; }
    public LocalDate getFecha() {
return fecha; }

    @Override
    public String toString() {
        return cliente + " - " +
articulo;
    }
}
```

OPCIÓN B · CON **LOMBOK**

entidades/Venta.java

```
package ar.edu.utnfc.backend.entidades;

import java.time.LocalDate;
import lombok.Data;
import lombok.AllArgsConstructor;

// genera constructor, getters,
toString,
// equals y hashCode solo, sin
escribirlos
@Data
@AllArgsConstructor
public class Venta {
    private String cliente;
    private String articulo;
    private double precioUnitario;
    private int cantidad;
    private LocalDate fecha;
}
```

ANOTACIÓN	QUÉ GENERA
`@Data`	getters, setters, `toString()`, `equals()` y `hashCode()` de todos los atributos
`@AllArgsConstructor`	constructor con todos los atributos, en el mismo orden en que están declarados
`@ToString(callSuper = true)`	igual que arriba, pero incluye también los atributos heredados de la clase padre
`@EqualsAndHashCode(callSuper = true)`	lo mismo para `equals()` / `hashCode()`, incluyendo la herencia

> **No es obligatorio:** las dos opciones son 100% equivalentes para el parcial. Para usar Lombok hacen falta dos cosas: agregar la dependencia `org.projectlombok:lombok` al `pom.xml` (con `scope: provided`), y tener el **Extension Pack for Java**, que ya reconoce Lombok (antes hacía falta una extensión aparte). Si el editor igual marca error, recargá la ventana.

2 — servicios / ServicioVentas.java

Cada punto que pide el enunciado ("mostrar tal cosa", "calcular tal otra") es **un método nuevo acá**, no una `Venta` inventada a mano. El servicio ya recibe la lista completa, cargada del CSV real:

servicios/ServicioVentas.java

```
package ar.edu.utnfc.backend.servicios;

import ar.edu.utnfc.backend.entidades.Venta;
import java.util.List;

public class ServicioVentas {
    private final List<Venta> ventas;              // recibe la lista ya cargada del
CSV

    public ServicioVentas(List<Venta> ventas) {
        this.ventas = ventas;
    }

    // Punto 1 del enunciado — cada punto nuevo es un método más acá
    public void punto1() {
        double total = ventas.stream().mapToDouble(Venta::getTotal).sum();
        System.out.println("Ventas: " + ventas.size() + " — Total: $" + total);
    }

    // public void punto2() { ... }   // se arma igual, cuando aparezca el punto 2
}
```

3 — app / Main.java

`Main` no crea `Ventas` a mano: **siempre lee el CSV** (eso no cambia entre puntos) y después llama un método del servicio por cada punto del enunciado. Para resolver un punto nuevo se agrega un método en `ServicioVentas` y una línea acá — la lectura queda intacta:

app/Main.java

```
package ar.edu.utnfc.backend.app;

import ar.edu.utnfc.backend.entidades.Venta;
import ar.edu.utnfc.backend.servicios.LectorCsv;
import ar.edu.utnfc.backend.servicios.ServicioVentas;
import java.nio.file.Path;
import java.util.List;

public class Main {
    public static void main(String[] args) throws Exception {
        // 1. Leer el CSV — siempre igual, sea cual sea el punto que pida el
enunciado
        List<Venta> ventas = LectorCsv.leer(Path.of("data.csv"));
        ServicioVentas servicio = new ServicioVentas(ventas);

        // 2. Un método por cada punto pedido — se descomenta/agrega a medida que se
resuelve
        servicio.punto1();
        // servicio.punto2();
    }
}
```

Ejecutar: `mvn compile exec:java`

---

07 · Foco del parcial · Parte 1

Leer cualquier CSV

El archivo `data.csv` va en la **raíz del proyecto**, al lado del `pom.xml` (no dentro de `src/`). La lectura arranca en `app/Main.java`, que le pasa la ruta a un método de `servicios/LectorCsv.java` — ahí vive toda la lógica de parseo:

app/Main.java

```
Path archivo = Path.of("data.csv");             // al lado del pom.xml
List<Venta> ventas = LectorCsv.leer(archivo);   // toda la lectura pasa por acá
System.out.println("Filas cargadas: " + ventas.size());
```

1 — Mirar el separador

Abrí el archivo en un editor de texto (no Excel) y mirá el encabezado. No asumas coma: en CSVs reales aparece también `;`, `\t` o `|`.

data.csv (real, separado por coma)

```
Nombre Cliente,Apellido Cliente,EMail Cliente,Arista,Album,Genero,...
Ana,Pérez,ana.perez@ejemplo.com,Battlestar Galactica (Classic),"Battlestar Galactica (Classic), Season 1",...
```

otro.csv (real, separado por punto y coma)

```
name;category;year_published;designer;min_age;average_rating
Tal der Könige;Ancient;1992;Christian Beierer;12;6.60675
```

Ojo con `"...Classic), Season 1"`: tiene una coma dentro de comillas — si el separador es coma, hay que ignorar esa.

2 — Con qué leerlo: mejor BufferedReader

El CSV del parcial suele tener miles de filas. `Scanner` sirve (está en el apunte 11), pero tokeniza con expresiones regulares y es más lento con archivos grandes. Lo más práctico es leer línea por línea con `BufferedReader`, otra de las formas del apunte, igual que un `Iterator`.

HERRAMIENTA	COMILLAS	USO
`Scanner`	No	Sirve para archivos chicos; más lento con miles de filas
`BufferedReader` + regex	Sí	**Recomendado**
OpenCSV	Sí	Alternativa si preferís no tocar regex

3 — Lector con BufferedReader (recomendado)

El patrón de siempre: `BufferedReader` + `readLine()` en un `while`, y cada fila se le pasa cruda (`String[]`) al constructor de la entidad — ella arma sus propios tipos (`double`, `LocalDate`, etc.). Agregá este constructor a `Venta`:

entidades/Venta.java — agregar

```
// arma la Venta a partir de la fila cruda del CSV
public Venta(String[] datos) {
    this(datos[0], datos[1], datos[2],
         Double.parseDouble(datos[3]),
         Integer.parseInt(datos[4]),
         LocalDate.parse(datos[5]));
}
```

Y el lector, que detecta el separador solo mirando el encabezado:

servicios/LectorCsv.java

```
package ar.edu.utnfc.backend.servicios;

import ar.edu.utnfc.backend.entidades.Venta;
import java.io.BufferedReader;
import java.io.FileReader;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Pattern;

public class LectorCsv {

    private static final char[] CANDIDATOS = {',', ';', '\t', '|'};

    public static List<Venta> leer(Path archivo) throws Exception {
        List<Venta> ventas = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new
FileReader(archivo.toFile()))) {
            String encabezado = br.readLine();
            char separador = detectarSeparador(encabezado);

            // corta por el separador, pero no si está entre comillas
            Pattern patron = Pattern.compile(
                Pattern.quote(String.valueOf(separador)) + "(?=(?:[^\"]*\"[^\"]*\")*
[^\"]*$)");

            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.isBlank()) continue;
                String[] datos = patron.split(linea, -1);
                for (int i = 0; i < datos.length; i++) {
                    datos[i] = datos[i].replace("\"", "").trim();
                }
                ventas.add(new Venta(datos));
            }
        }
        return ventas;
    }

    // cuenta cuál separador candidato aparece más veces en el encabezado
    private static char detectarSeparador(String encabezado) {
        char mejor = ',';
        long max = -1;
```

```
        for (char c : CANDIDATOS) {
            long n = encabezado.chars().filter(ch -> ch == c).count();
            if (n > max) { max = n; mejor = c; }
        }
        return mejor;
    }
}
```

Se llama igual que antes: `LectorCsv.leer(archivo)` desde `Main`. Si el CSV real no tiene comillas, se puede simplificar el `Pattern` por un `linea.split(String.valueOf(separador))` directo.

4 — Alternativa con una librería: OpenCSV

Mismo resultado, sin regex, agregando una dependencia:

pom.xml — dependencia

```
<dependency>
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>5.9</version>
</dependency>
```

servicios/LectorCsv.java (variante OpenCSV)

```
var parser = new CSVParserBuilder().withSeparator(separador).build();
try (CSVReader reader = new CSVReaderBuilder(new FileReader(archivo.toFile()))
        .withCSVParser(parser)
        .withSkipLines(1)                    // saltea el encabezado
        .build()) {
    String[] datos;
    while ((datos = reader.readNext()) != null) {
        ventas.add(new Venta(datos));
    }
}
```

5 — Menú: lo fijo vs. lo que se ajusta según el CSV

Para armar un menú (1, 2, 3…) donde cada opción resuelve un punto del enunciado, la lectura sigue siendo la misma de más arriba (`LectorCsv.leer(...)`, que ya convierte cada fila en un objeto real) — lo único que cambia según el punto es qué hacés con esa lista ya armada:

PARTE	CAMBIA SEGÚN EL CSV / EL PUNTO?
`LectorCsv.leer(...)` — leer y armar los objetos	No — se escribe una vez y no se vuelve a tocar
Qué mostrás o calculás con esos objetos (usando getters)	Sí — un método nuevo por cada punto del enunciado

app/Main.java

```
package ar.edu.utnfc.backend.app;

import ar.edu.utnfc.backend.entidades.Venta;
import ar.edu.utnfc.backend.servicios.LectorCsv;
import java.nio.file.Path;
import java.util.List;
import java.util.Scanner;

public class Main {

    // ---- SE AJUSTA acá: qué mostrás, usando los getters de la entidad ----
    static void mostrarTabla(List<Venta> ventas) {
        System.out.printf("%-15s %-15s %-10s%n", "Cliente", "Articulo", "Total");
        for (Venta v : ventas) {
            System.out.printf("%-15s %-15s %-10.2f%n", v.getCliente(),
v.getArticulo(), v.getTotal());
        }
    }

    public static void main(String[] args) throws Exception {
        // ---- FIJO: se lee una sola vez, antes del menú, y no se vuelve a tocar ---
-
        List<Venta> ventas = LectorCsv.leer(Path.of("data.csv"));

        Scanner sc = new Scanner(System.in);
        int opcion;
        do {
            System.out.println("\n1) Mostrar CSV\n0) Salir");
            opcion = Integer.parseInt(sc.nextLine());

            switch (opcion) {
                case 1 -> mostrarTabla(ventas);
                // case 2 -> ...   próximo punto del enunciado, va acá
            }
        } while (opcion != 0);
    }
}
```

Por qué no usar `String[]` hasta el final: si te quedás con las filas crudas (`f[0]`, `f[1]`...) para todo el programa, cada método nuevo tiene que volver a acordarse qué posición es cada columna, y un error de índice es invisible hasta que corre mal. Convirtiendo cada fila en objeto (`Venta`, o `Aereo`/`Terrestre`/`Maritimo` si el enunciado tiene herencia) una sola vez, en el lector, el resto del programa usa `v.getCliente()`, `v.getTotal()` — nombres, no posiciones. Los títulos ("Cliente", "Total"...) siguen siendo texto suelto que vos tipeás, pero ahora al lado va un getter con nombre, no un índice a ciegas. `%n` en `printf` es un salto de línea, igual que `\n`.

Cada punto nuevo del enunciado (búsqueda, filtro, cálculo) es un método más, sumado con su propio `case` en el `switch` — la lectura queda intacta siempre.

---

08 · Foco del parcial · Parte 2

Búsquedas, filtros y agregaciones

Imports: `import java.util.Comparator;` y, para `groupingBy`, `counting`, `summingDouble`, `toMap` y `partitioningBy`, `import static java.util.stream.Collectors.*;` (o escribir `Collectors.groupingBy(...)`). Todas empiezan con `lista.stream()`.

Filtro exacto

```
.filter(v ->
v.getCategoria().equals("Rock"))
```

Contiene texto

```
.filter(v ->
v.getArticulo().toLowerCase().contains("m
ouse"))
```

Rango numérico

```
.filter(v -> v.getTotal() >= 100 &&
v.getTotal() <= 500)
```

Rango de fechas

```
.filter(v ->
!v.getFecha().isBefore(desde) &&
!v.getFecha().isAfter(hasta))
```

Ordenar

```
.sorted(Comparator.comparing(Venta::getTo
tal).reversed())
```

Top-N

```
.sorted(Comparator.comparing(Venta::getTo
tal).reversed()).limit(10)
```

Contar por categoría

```
.collect(groupingBy(Venta::getCategoria,
counting()))
```

Sumar por categoría

```
.collect(groupingBy(Venta::getCategoria,
summingDouble(Venta::getTotal)))
```

Promedio por categoría

```
.collect(groupingBy(Venta::getCategoria, averagingDouble(Venta::getTotal)))
```

Sin duplicados

```
.map(Venta::getCliente).distinct().toList()
```

Máximo / mínimo

```
.max(Comparator.comparing(Venta::getTotal))
```

Primero que cumple algo

```
.filter(v -> v.getCliente().equals("Ana")).findFirst()
```

Existencia (any/all/none)

```
.anyMatch(v -> v.getTotal() > 1000)
```

Indexar en Map

```
.collect(toMap(Venta::getCliente, v -> v, (a,b) -> a))
```

Partición (dos grupos)

```
.collect(partitioningBy(v -> v.getTotal() > 100))
```

Ejemplos reales (recuperatorio anterior)

Top 10 facturas de menor monto, de mayor a menor:

ServicioFacturas.java

```
List<Factura> top10 = facturas.stream()
        .sorted(Comparator.comparing(Factura::getTotal))
        .limit(10)
        .sorted(Comparator.comparing(Factura::getTotal).reversed())
        .toList();
```

Top 5 canciones más vendidas, agrupando track + género + media type:

ServicioVentas.java

```
record ResumenTrack(String track, String genero, String mediaType) {}

Map<ResumenTrack, Long> ventasPorTrack = items.stream()
        .collect(groupingBy(
                i -> new ResumenTrack(i.getTrack().getNombre(),
                        i.getTrack().getGenero().getNombre(),
                        i.getTrack().getMediaType().getNombre()),
                summingLong(InvoiceItem::getCantidad)));

List<Map.Entry<ResumenTrack, Long>> top5 = ventasPorTrack.entrySet().stream()
        .sorted(Map.Entry.<ResumenTrack, Long>comparingByValue().reversed())
        .limit(5)
        .toList();
```

09 · Referencia rápida

Comandos de Maven

COMANDO	QUÉ HACE
`mvn -v`	Versión de Maven / JDK
`mvn clean`	Borra `target/`
`mvn compile`	Compila
`mvn exec:java`	Ejecuta el `mainClass` del pom.xml
`mvn clean compile exec:java`	Los tres juntos — el más usado
`mvn package`	Genera el `.jar`

10 · Antes de entregar

Checklist

- [ ] Separador confirmado mirando el archivo, no adivinado.
- [ ] Separador en una sola variable, no repetido a mano.
- [ ] Comillas con separador adentro → `split()` con el regex de comillas, no `split()` a secas.
- [ ] `.trim()` antes de parsear números/fechas.

- [ ] Filas vacías/incompletas descartadas si el enunciado lo pide.
- [ ] `mvn clean compile exec:java` corre bien desde cero.
- [ ] `app` / `entidades` / `servicios` separados.

Basado en apuntes de cátedra (Maven, JDK, Colecciones/Iteradores, Streams, Procesamiento de CSV) y un enunciado real de Backend de Aplicaciones — UTN FRC.